Telecommunications Network

Practice 1

Requirements

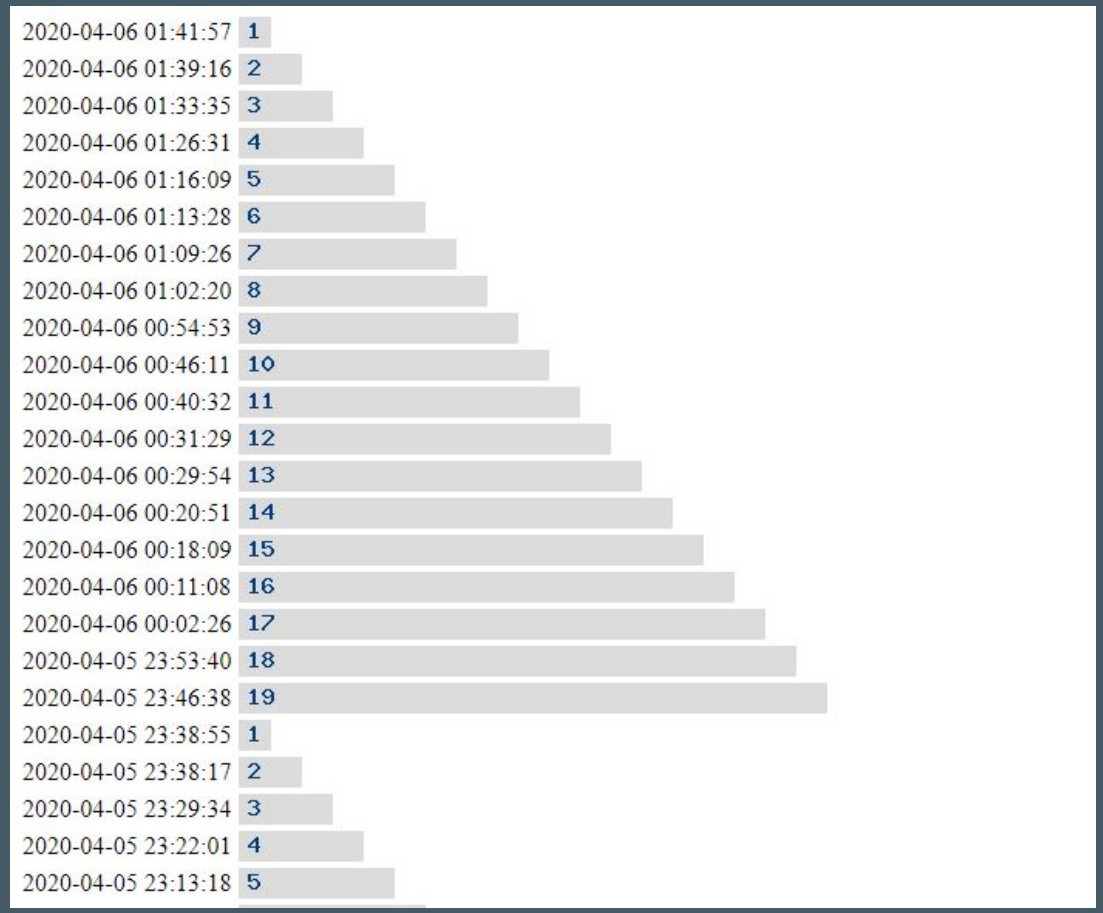
- Maximum 4 absences
- Final grade:
  - Socket assignments (4 pcs, min. 50%)
  - Socket exam (min. 50%)
  - Kathará exam (min. 50%)

Socket assignments

- Programming and simulation task
- The deadline is 3 weeks
- It must be submitted via TMS, which will evaluate and check for code similarity
- The result is put in the comment field, the tester is run periodically
- Submitting plagiarized code is considered fraud and we act in accordance with university rules

Deadline Panic

Don’t procrastinate!



The tester’s processing queue, around deadline

Grade limits

$$\% = \frac{assignments\%}{3} + \frac{socket\%}{3} + \frac{kathara\%}{3}$$

Percent	Grade
0 - 49%	Fail (1)
50 - 59%	Sufficient (2)
60 - 74%	Fair (3)
75 - 84%	Good (4)
85 - 100%	Excellent (5)

Topics

- Python basics
- Socket programming
- CRC, encodings, MD5
- Firewalls: IPtables
- Wireshark/tcpdump traffic analysis
- Kathará, network characteristics, basic tools: traceroute, ping
- MAC learning, STP, ARP, routing configs
- Port forwarding, VLAN setup
- Tunneling solutions IPv4/IPv6

Python history

Guido Van Rossum, in ’96:

“Over six years ago, in December 1989, I was looking for a”hobby” programming project that would keep me occupied during the week around Christmas. My office … would be closed, but I had a home computer, and not much else on my hands. I decided to write an interpreter for the new scripting language I had been thinking about lately: a descendant of ABC that would appeal to Unix/C hackers. I chose Python as a working title for the project, being in a slightly irreverent mood (and a big fan of Monty Python’s Flying Circus).”



Python

- Designed for easy learning
  - Clean, simple syntax, few, short keywords
- Portable:
  - Runs on almost everything (linux, windows, mac, Raspberry Pi,)
- Uses whitespace to separate blocks
  - It should be that way anyway for readability
- Variables don’t need to be declared
  - But not a type-independent language
- Versions
  - python 3.x (on lab machines: python, py)
  - python2 DEPRECATED

Zen of python

```
# import this
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
```

Python REPL (Read-Eval-Print Loop)

Starting up

```
$ python
$ py # on lab machines
```

Hello World:

```
print("Hello world!")
user_name = "Foo"
print("Hello " + user_name)
user_age = 25
print("You are " + str(user_age) + " years old.")
```

```
Hello world!
Hello Foo
You are 25 years old.
```

Simple arithmetics

```
print(10+2)
print(2*2)
print(3**2)
print(10%2)
```

```
12
4
9
0
```

Mathematical rounding

```
import math
print(math.floor(3.8))
print(round(3.8))
print(round(3.8,1))
```

```
3
4
3.8
```

Variables

```
a = 42
b = 32
c = a + b
print(c)
c = 'valami'
print(a+c)
```

74

TypeError: unsupported operand type(s) for +: 'int' and 'str'

String operations

It doesn’t matter whether you use ' or ", just use it consistently

```
print("alma".upper())
print("LO" in "Hello".upper())
print("Decimal Number: %d, Float: %f, String: %s" % (12,33.4,"almafa"))
x = 42
print(f"x == {x}")
```

```
ALMA
True
Decimal Number: 12, Float: 33.400000, String: almafa
x == 42
```

List

```
players = [12, 31, 27, '48', 54]
print(players)
print(players[0])
print(players[-1])
print(players + [22, 67])
print(len(players))
```

```
[12, 31, 27, '48', 54]
12
54
[12, 31, 27, '48', 54, 22, 67]
5
```

List

```
players = [12, 31, 27, '48', 54]
players.append(89)
print(len(players))
print(players[2:])
```

```
6
[27, '48', 54, 89]
```

Tuple – immutable List

```
players = (12, 31, 27, '48', 54)
print(players[2:])
players[2] = 'alma'
del players[2]
```

(27, '48', 54)

TypeError: 'tuple' object does not support item assignment

Set

```
mylist = [8,3,2,3,2,4,6,8,2]
myset = set(mylist)
print(mylist)
print(myset)
mysortedlist = sorted(mylist)
print(mysortedlist)
```

```
[8, 3, 2, 3, 2, 4, 6, 8, 2]
{2, 3, 4, 6, 8}
[2, 2, 2, 3, 3, 4, 6, 8, 8]
```

Dictionary

```
team = {
  91: "Ayers, Robert",
  13: "Beckham Jr,",
  3:  "Brown, Josh",
  54: "Casillas, Jonathan",
  21: "Collins, Landon"
}
print(len(team))
team[3] = "Chihiro"
print(91 in team)
print("alma" in team)
```

```
5
True
False
```

Dictionary

```
team = {
  91: "Ayers, Robert",
  13: "Beckham Jr,",
  3:  "Brown, Josh",
  54: "Casillas, Jonathan",
}
print(team.keys())
print(team.values())
```

```
dict_keys([91, 13, 3, 54])
dict_values(['Ayers, Robert', 'Beckham Jr,', 'Brown, Josh', 'Casillas, Jonathan'])
```

If

```
if 100 in team:
  print("Yes, 100 is in the team")
elif 76 in team:
  print("100 is not in the team, but 76 is in it...")
else:
  print("Both 100 and 76 are not in the team")
```

Both 100 and 76 are not in the team

For loop

```
mylist = [3,65,2,77]
for i in mylist:
  print("Element:", i)

# Write the numbers in ascending order with two increments!
for i in range(2,10,2): # (start, stop, step)
  print(i)

for (k,v) in team.items():
  print("Player name: %s; #: %d" % (v,k))
# ...
```

```
Element: 3
Element: 65
Element: 2
Element: 77
2
4
6
8
Player name: Ayers, Robert; #: 91
Player name: Beckham Jr,; #: 13
Player name: Brown, Josh; #: 3
Player name: Casillas, Jonathan; #: 54
```

While loop

```
i=1
while i<10:
  print(i)
  i+=1
```

```
1
2
3
4
5
6
7
8
9
```

Running a Python script

```
# vim test.py
#!/usr/bin/env python3
x = 1
for i in range(1, 5):
  x+=i # no operator++
  print(x, i, 'alma', 'x*x = %d' % (x*x))
  print(str(i) + " alma")
```

```
2 1 alma x*x = 4
1 alma
4 2 alma x*x = 16
2 alma
7 3 alma x*x = 49
3 alma
11 4 alma x*x = 121
4 alma
```

Run:

```
$ ./test.py # chmod +x test.py
$ python test.py
$ py test.py # on lab machines
```

Functions

```
def is_even(num):
  if (num % 2) == 0:
    return True
  else:
    return False

for i in range(1,10):
  if (is_even(i)):
    print("Num:"+str(i))
print("Done")
```

```
Num:2
Num:4
Num:6
Num:8
Done
```

Functions

```
def complex(x):
  return x**2, x**3, x**4

print(complex(2))
a, b, c = complex(2)
print(a,b,c)
_, rv, _ = complex(2)
print(rv)
```

```
(4, 8, 16)
4 8 16
8
```

Lambda Functions

```
is_even = lambda num: (num % 2) == 0
is_even_2 = lambda num: True if (num % 2) == 0 else False
for i in range(1,10):
  if (is_even(i)):
    print("Num:"+str(i))
print("Done")
```

```
Num:2
Num:4
Num:6
Num:8
Done
```

List, Dict, Tuple comprehension

```
print([ x*x for x in range(10) ])
print({ x:x*x for x in range(5) })
print({ x:x*x for x in range(5) if x!=2 })
print(tuple( x*x for x in range(3) ))
```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
{0: 0, 1: 1, 3: 9, 4: 16}
(0, 1, 4)
```

Map

```
def fahrenheit(T):
  return ((float(9)/5)*T + 32)

def celsius(T):
  return (float(5)/9)*(T-32)

temperatures = (36.5, 37, 37.5, 38, 39)
F = map(fahrenheit, temperatures)
C = map(celsius, F)
temperatures_in_F = list(map(fahrenheit, temperatures))
temperatures_in_C = list(map(celsius, temperatures_in_F))
print(temperatures_in_F)
print(temperatures_in_C)
```

```
[97.7, 98.60000000000001, 99.5, 100.4, 102.2]
[36.5, 37.00000000000001, 37.5, 38.00000000000001, 39.0]
```

Filter

```
fibonacci = [0,1,1,2,3,5,8,13,21,34,55]
odd_numbers = list(filter(lambda x: x % 2, fibonacci))
print(odd_numbers)
```

```
[1, 1, 3, 5, 13, 21, 55]
```

File operations

```
f = open("demofile.txt", "r")
print(f.read())
print(f.readline())
for x in f:
  print(x)
f.close()
```

```
with open("alma.txt", "r") as f:
   for line in f:
      print(line.strip().split(","))
```

```
f = open("demofile.txt", "w") # w-write, a-append
f.write("lorem ipsum")
```

Read from standard input

```
x = input("Please enter a number:")
# type of x is always str!
print("Entered number:", x)
```

Command line arguments

```
import sys
print(sys.argv[0]) # name of the script
print(sys.argv[1]) # first parameter
print(sys.argv[2]) # second parameter
# ...
```

Classes

```
class Student:
  name = ''
  zhpoint = 0
  def __init__(self,_name,_point):
    self.name =  _name
    self.zhpoint = _point

  def __str__(self): # human readable
    return  f"{self.name}: {self.zhpoint} point"

  def __repr__(self): # machine readable
    return self.name+"("+str(self.zhpoint)+")"

p = Student("Ford",20)
print(p)
print([p])
```

```
Ford: 20 point
[Ford(20)]
```

Import vs main()

importtest.py

```
def main():
    print("Inside main")
if __name__ == "__main__":
    print("Executed as script")
    main()
```

testimport.py:

```
import importtest
importtest.main()
```

or:

```
from importtest import main
main()
```

Output:

```
$ python3 importtest.py
Executed as script
Inside main
$ python3 testimport.py
Inside main
```

JSON - JavaScript Object Notation

See: https://www.rfc-editor.org/rfc/rfc8259

```
{
   "firstName": "Jane",
   "lastName": "Doe",
   "hobbies": ["running", "sky diving", "singing"],
   "age": 35,
   "children": [
      {
         "firstName": "Alice",
         "age": 6
      },
      {
         "firstName": "Bob",
         "age": 8
      }
   ]
}
```

JSON & Python

Write dictionary as a JSON file

```
import json
data = {
    "president": {
        "name": "Zaphod Beeblebrox",
        "species": "Betelgeusian"
    }
}
with open("data_file.json", "w") as write_file:
    json.dump(data, write_file)
```

Dictionary as JSON formatted string

```
json_string = json.dumps(data)
```

Serialization →

Python	JSON
dict	object
list, tuple	array
str	string
int, float	number
True	true
False	false
None	null

Deserialization ←

Python	JSON
dict	object
list	array
str	string
int	number(int)
float	number(real)
True	true
False	false
None	null

JSON & Python

Reading a JSON object from a JSON file

```
import json
with open("data_file.json", "r") as read_file:
    data = json.load(read_file)
    print(data["president"]["name"])
```

JSON & Python

Read a JSON object from a string in JSON format

```
import json
json_string = """
{
    "researcher": {
        "name": "Ford Prefect",
        "species": "Betelgeusian",
        "relatives": [
            {
                "name": "Zaphod Beeblebrox",
                "species": "Betelgeusian"
            }
        ]
    }
}
"""
data = json.loads(json_string)
for rel in data["researcher"]["relatives"]:
        print("Name: %s (%s)" % ( rel["name"], rel["species"] ) )
```

Practice I.

Let’s write a function that outputs whether the year given as the parameter is a leap year. Let’s read the years from a file! A year is a leap year if it’s divisible by four, but not if it’s divisible by a hundred, unless it’s divisible by four hundred.
Examples:

- leap year: 1992, 1996, 2000, 2400
- not a leap year: 1993, 1900

Practice II.

Let’s write a script that calculates how many points are required on the Kathará exam for each grade. The input should be a json file containing the maximum score and the minimal result for the submissions, socket and the Kathará exam and the achieved results for the submissions and the socket exam. The output should be the minimum point required for each grade. (There are no fractions!)

Input:

```
{
  "homework": {
    "max": 4,
    "point": 2,
    "min_percent": 0.5
  },
  "socket": {
    "max": 100,
    "point": 76,
    "min_percent": 0.5
  },
  "kathara": {
    "max": 10,
    "min_percent": 0.5
  }
}
```

Output:

```
$ python zh_grade.py
2: 5
3: 6
4: 10
5: Nope
```